

Built-in Template Automation Implementation Task List

Scope

Implementation breakdown derived from [docs/specs/built-in-template-automation-test-suite-requirements.md](#).

Milestone A: Runner Contract and Discovery

- ☐ Add a dedicated local runner entry point for template automation suite.
- ☐ Add CLI parameters:
 - `-Mode` (`AllTemplates`, `SelectedTemplates`)
 - `-TemplateIds` (single/multiple IDs)
 - `-Distro`
 - `-OutputRoot`
 - `-IncludeCapabilityGated`
 - `-DryRun`
 - `-AllowCiOverride`
- ☐ Implement metadata-driven discovery from `config/templates.json`.
- ☐ Implement unknown template ID validation with fail-fast diagnostics.
- ☐ Implement planned-execution preview for `-DryRun`.

Milestone B: Local-Only Guard and Environment Preflight

- ☐ Add local-only guard (default skip when CI indicators are detected).
- ☐ Add explicit override path guarded by `-AllowCiOverride`.
- ☐ Capture environment snapshot at run start:
 - `wsl --status`
 - `wsl --version`
 - `wsl --list --verbose`
- ☐ Add distro existence and readiness checks for selected target distro.
- ☐ Add per-template capability checks (GPU/systemd/Docker Desktop integration).

Milestone C: Execution Orchestration

- ☐ Implement `AllTemplates` execution loop over discovered built-in templates.
- ☐ Implement `SelectedTemplates` execution loop preserving user-specified order.
- ☐ Invoke existing template apply flow for each test item.
- ☐ Add per-item timeout, cancellation, and retry policy hooks.
- ☐ Persist per-item raw execution logs.

Milestone D: Probe Library and Status Classification

- ☐ Create shared probe helpers for WSL command execution and output capture.
- ☐ Implement language-family probe sets:
 - .NET (`dotnet --list-sdks`)
 - Node.js (`nvm`, `node`, package managers)

- Python (`pyenv`, `python`, optional tools)
- Java (`sdk`, `java -version`)
- Rust (`rustc`, `cargo`, channel)
- Go (`go version`)
- ☐ Implement scenario-family probe sets:
 - Container runtime (docker/podman)
 - Kubernetes local (`kubect1 get nodes` + cluster tool)
 - Database local stack (service/health/connectivity)
 - AI/ML profiles (CPU/GPU prerequisite + smoke command)
- ☐ Implement unified result classifier:
 - `Pass`
 - `Fail`
 - `Blocked` (capability missing)

Milestone E: Result Persistence and Documentation

- ☐ Generate machine-readable XML result file (`NUnitXml` or `JUnitXml`).
- ☐ Generate run manifest JSON with per-item details.
- ☐ Generate markdown summary at:
 - `docs/development/testing/results/<yyyymmdd>/<run-id>/summary.md`
- ☐ Maintain historical index at:
 - `docs/development/testing/results/index.md`
- ☐ Ensure summary contains pass/fail/blocked counters and environment snapshot.

Milestone F: Developer Experience and Safety

- ☐ Add clear startup banner indicating local-only intent and expected runtime cost.
- ☐ Add command examples for full run and selective run in `tests/README.md`.
- ☐ Add troubleshooting guide section for common `Blocked` reasons.
- ☐ Ensure logs never include secrets or sensitive host data.

Milestone G: Final Validation and Handover

- ☐ Run one full-catalog dry run to verify discovery contract.
- ☐ Run one selective validation (single template ID).
- ☐ Run one selective validation (multiple template IDs).
- ☐ Verify XML + JSON + markdown outputs are generated and linkable.
- ☐ Document known limitations and open follow-up tasks.

Suggested Delivery Order

1. Milestone A-B (runner contract, guard, preflight)
2. Milestone C-D (execution + probes + classification)
3. Milestone E (artifact and docs output)
4. Milestone F-G (DX hardening and acceptance readiness)